



Lesson #8:

CONTINUOUS DELIVERY & DEPLOYMENT (CD)

Lesson objectives

1 Understand the CD process

Master the continuous delivery (CD pipeline) and continuous deployment processes.

2 Package applications with Docker

Know how to package applications as images with Docker.

3 Apply CD with Jenkins

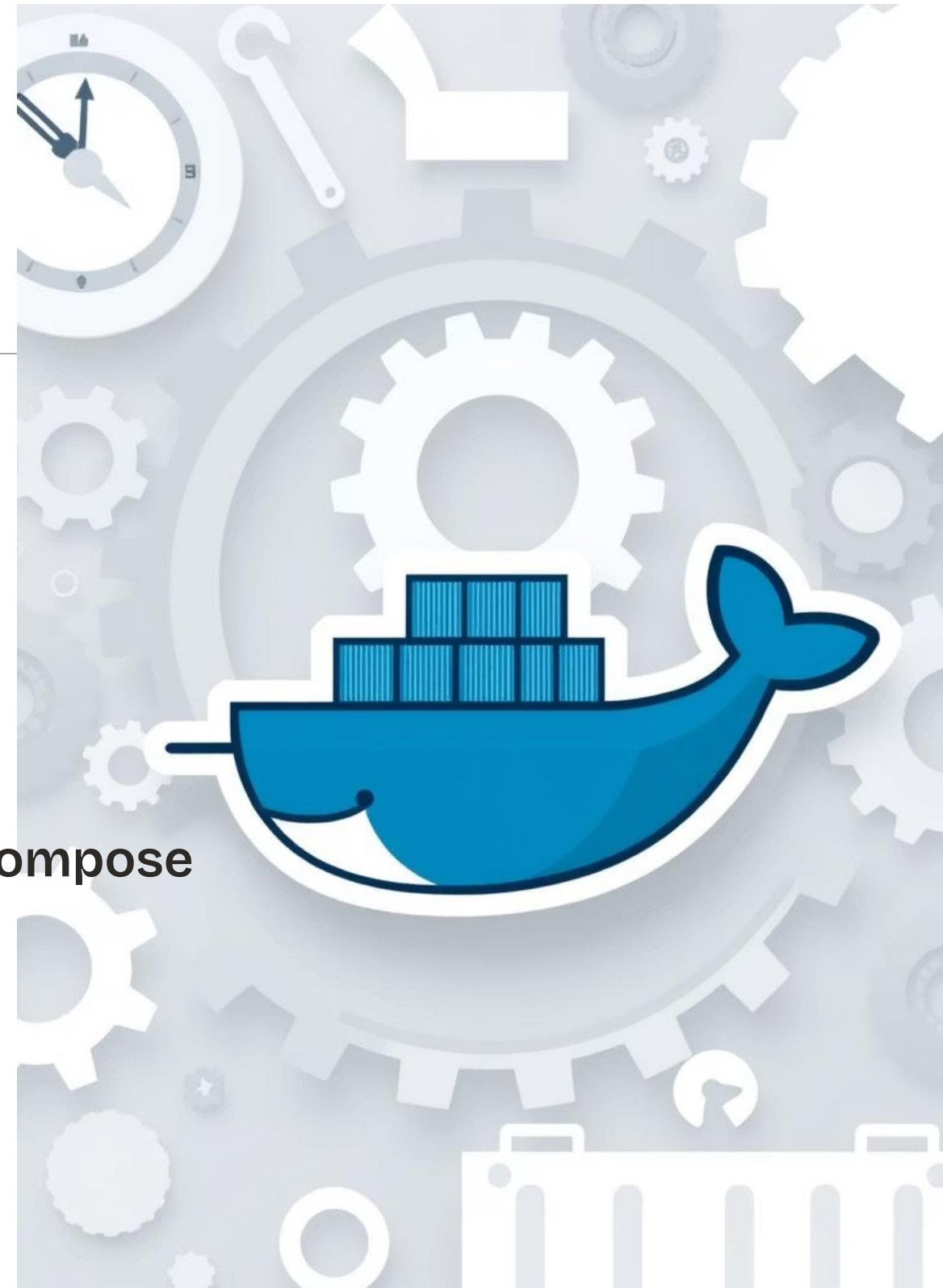
Combine Jenkins with Docker to build an effective CD process.





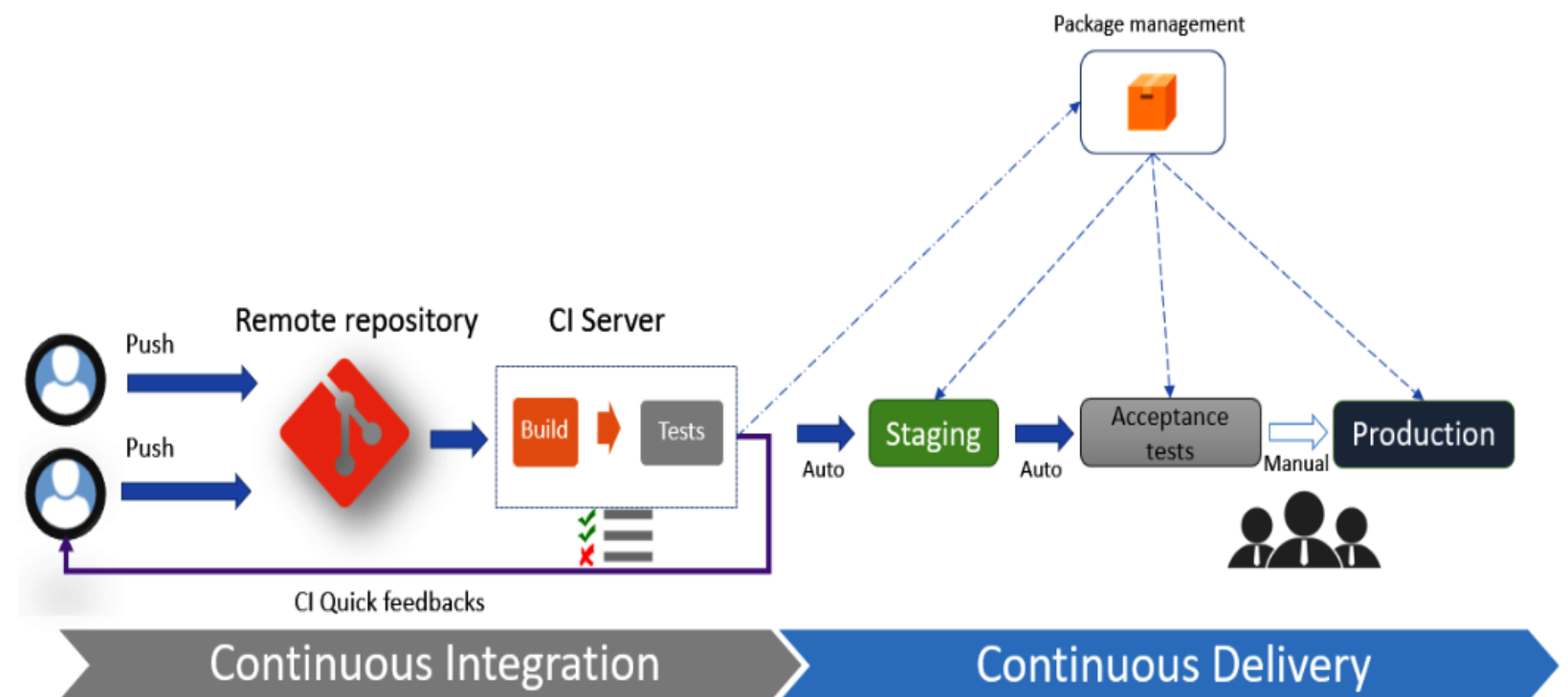
Main content

- 1 Continuous delivery & deployment process - CD
- 2 Package applications with Docker
- 3 Using Docker with Command Line Tools & Docker Compose
- 4 CD practice with Jenkins



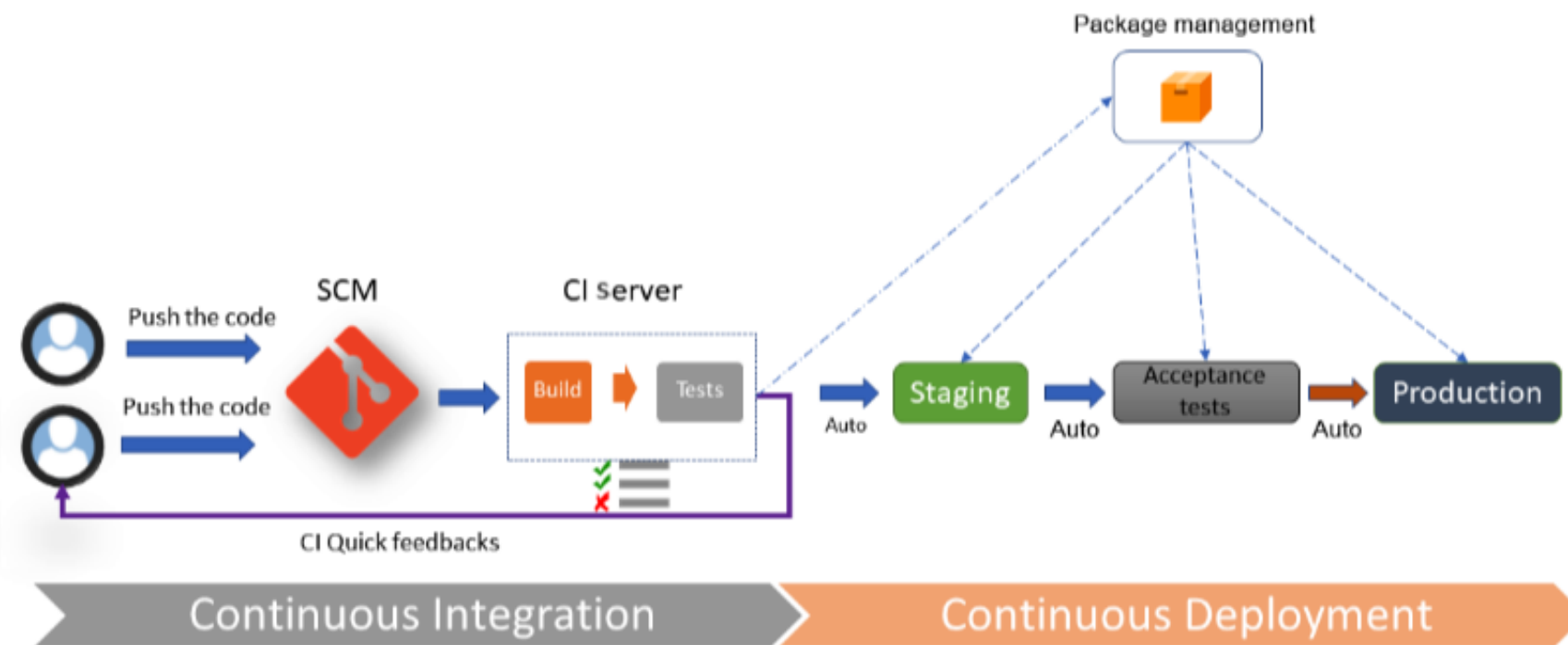
1. Continuous Delivery (CD) Process

- ❑ Continuous Delivery Process = CI + deploy application to deployment environments, but deploying to the end-user environment (Prod) is still manual.



1. Continuous Deployment Process

The continuous deployment pipeline



Continuous Deployment = CI + CD, but automates deployment to the end-user environment (Prod).



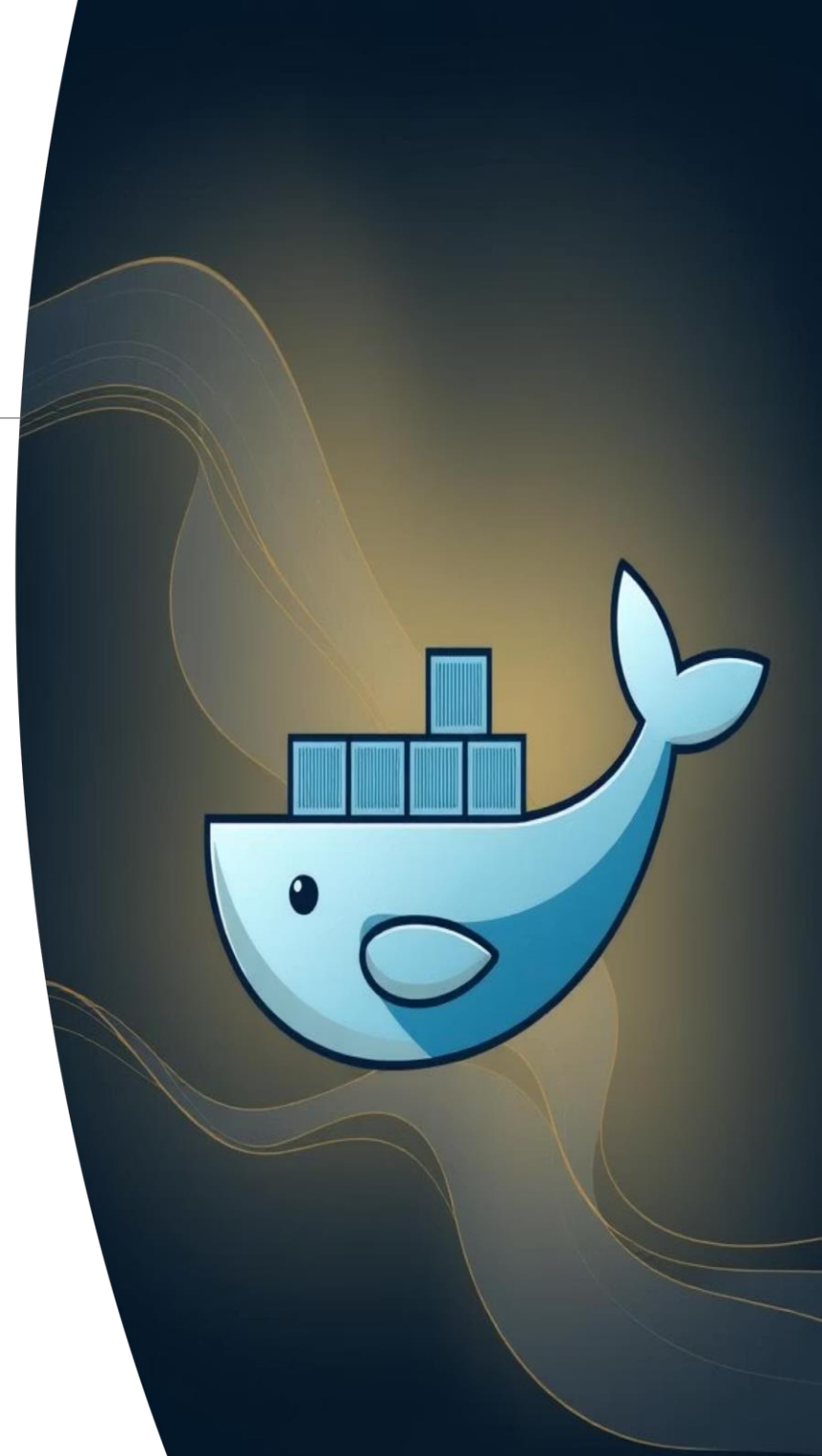
2. Package applications with Docker

- a) Introduction**
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands
- e) Key elements of Docker
- f) Main/core Docker directives
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container of the image
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to run a test



a. Docker – Introduction

- ❑ In recent years, a particular technology has attracted attention in the market, on social networks and at events, namely Docker.
- ❑ Docker is an open source containerization tool from 2013, which allows isolating applications from the server system.
 - Helps applications become mobile, can be tested on the developer's workstation and deployed to the production environment without worrying about runtime dependencies.
 - It is a cross-platform tool, can be installed on Windows, Linux, macOS, ...and is available by default on some cloud providers such as AWS, Azure, ...





a. Docker – Introduction



Container

A system that embeds the application with its dependencies, containing only a lightweight operating system and the necessary elements.



Comparison with VM

Unlike virtual machines (VMs) that contain a full operating system, containers rely on the guest operating system (the container only contains a lightweight operating system (OS) and the elements necessary for that operating system such as system libraries, binary files and code dependencies).



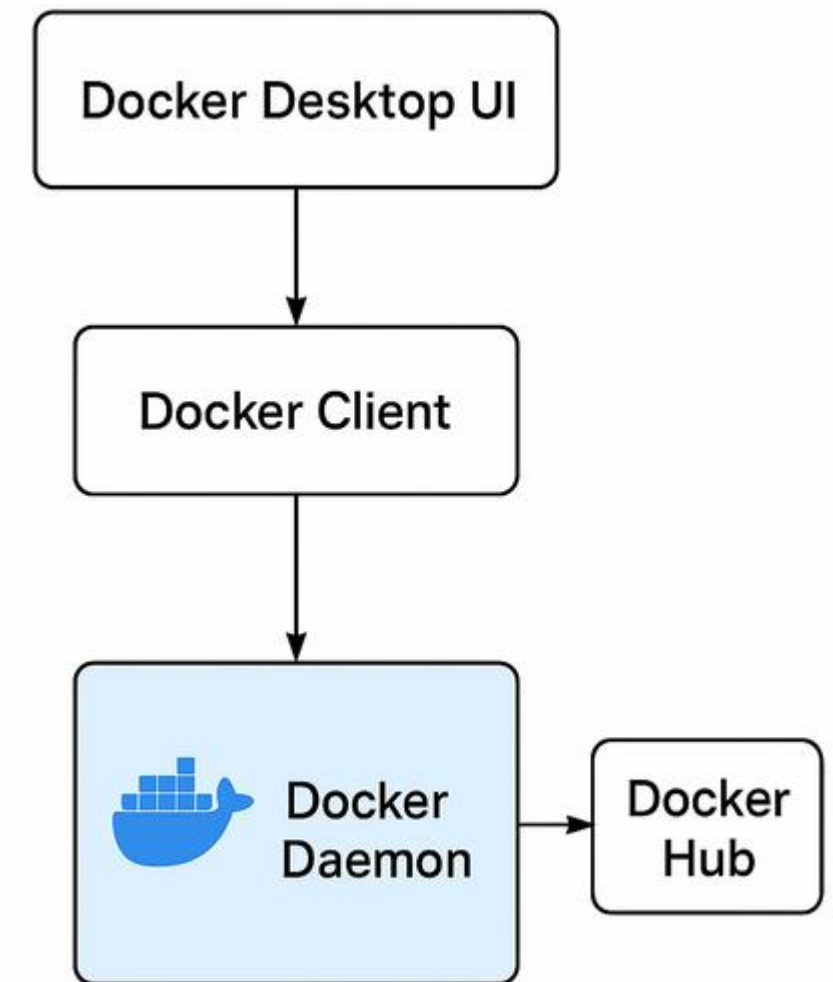
Portability

Docker makes applications portable by building images that are easy to deploy across different environments without compatibility issues.

a. Docker – Introduction

- ❑ To operate, Docker needs the following elements:
- Docker Desktop UI: manage containers
 - Docker client: Is a command line tool (docker) that you use to give commands, for example: docker run, docker build, ...
 - Docker daemon: Is the "engine" of Docker, responsible for: Managing containers, images, networks, volumes; Executing commands from the Docker Client.
 - Docker Registry: This is a public repository (Docker Hub) or a private repository of Docker images (account registration required)

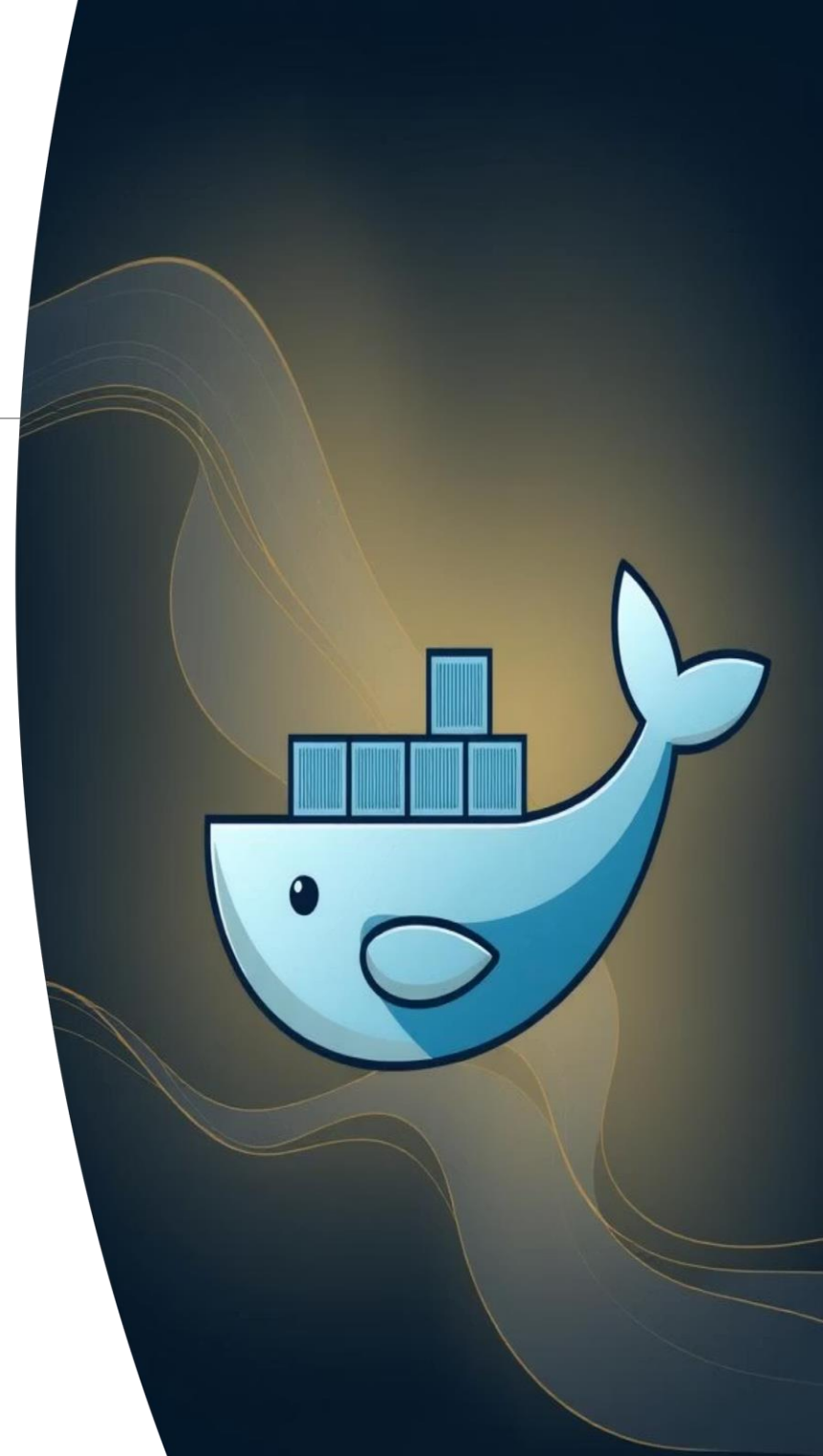
⇒ In Docker Desktop, Docker Client and Docker Daemon are available → run directly on Desktop (Windows /Mac/Linux).
Docker Desktop only connects to Docker Hub by default





2. Package applications with Docker

- a) Introduction
- b) Register & Explore Docker Hub**
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands
- e) Key elements of Docker
- f) Main/core directives of Docker
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container from the image
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to test run





b. Register & Explore Docker Hub

Docker Hub is a public space (registry) containing over 2 million public Docker images from companies, communities, and individual users.

To register, visit <https://hub.docker.com/>, fill in the information and click "Sign Up". This account allows us to upload arbitrary images and download Docker Desktop.

After logging in, we can explore the available images in the "Explore" section, with filters to search for official images, from verified publishers, or certified by Docker (Screenshot – Slide below)

The screenshot shows the Docker Hub website's registration page. At the top, there's a navigation bar with the Docker Hub logo, a search bar, and links for 'Explore', 'Pricing', 'Sign In', and 'Sign Up'. A banner on the left reads 'Build and Ship any Application Anywhere' with a subtext: 'Docker Hub is the world's easiest way to create, manage, and deliver your teams' container applications.' On the right, a 'Get Started Today for Free' form is visible. It includes a link for 'Already have an account? Sign In', input fields for 'Docker ID', 'Email', and 'Password', a checkbox for 'Send me occasional product updates and announcements', a CAPTCHA 'I'm not a robot' checkbox, and a blue 'Sign Up' button. At the bottom of the form, it states: 'By creating an account, you agree to the Subscription Service Agreement, Privacy Policy and Data Processing Terms.'

Registered with account ptthuong@ictu.edu.vn



b. Register & Explore Docker Hub

 Explore My Hub 

Search Docker Hub CtrlK     P

 **phamthuongdocker**
Docker Personal 

 **Repositories**

 Collaborations

 **Settings** 

Default privacy

Notifications

 Billing

 **Usage** 

Pulls

Storage

Repositories

All repositories within the **phamthuongdocker** namespace.

All content 

 [Create a repository](#)

Name	Last Pushed 	Contains	Visibility	Scout
phamthuongdocker/demobook	about 2 months ago	IMAGE	Public	Inactive

1-1 of 1  



b. Register & Explore Docker Hub

hub

Explore

My Hub

Search Docker Hub

CtrlK

?

🔔

🌙

☰

P

Filter by (1) [Clear All](#)

Products

☒ Images ✓

☐ Extensions

☐ Plugins

Trusted content

☐ Docker Official Image ⓘ

☒ Verified Publisher ⓘ ✓

☐ Sponsored OSS ⓘ

Categories

☐ Networking

1 - 28 of 11,443,319 available images.

Images x

IMAGE

mcp/fetch ✓
mcp

Fetches a URL from the internet and extracts its contents as markdown

Pulls500K+

Stars33

Last Updated2 months

IMAGE

mcp/slack ✓
mcp

Interact with Slack Workspaces over the Slack API.

Pulls100K+

Stars19

Last Updated2 months

IMAGE

mcp/time ✓
mcp

Time and timezone conversion capabilities

Pulls100K+

Stars5

Last Updated2 months

IMAGE

mcp/postgres ✓
mcp

Connect with read-only access to PostgreSQL databases. This server...

Pulls100K+

Stars25

Last Updated2 months

IMAGE

ai/qwen3 ✓
Docker

Qwen3 is the latest Qwen LLM, built for fast inference with reasoning

Pulls100K+

Stars19

Last Updated2 months

IMAGE

ai/llama3.2 ✓
Docker

Solid LLaMA 3 update, reliable for coding chat and QA tasks

Pulls100K+

Stars19

Last Updated2 months

IMAGE

ai/deepseek-r1-dist... ✓
Docker

Distilled LLaMA by DeepSeek, fast and optimized for real-world tasks

Pulls100K+

Stars5

Last Updated2 months

IMAGE

mcp/docker ✓
mcp

MCP smart gateway used by the AI Tool Catalog

Pulls100K+

Stars25

Last Updated2 months

Suggested

University of Information Technology and Communications

b. Register & Explore Docker Hub

- Languages & frameworks
- Integration & delivery
- Message queues
- API management
- Internet of things
- Machine learning & AI
- Developer tools
- Data science
- Web servers
- Operating systems
- Content management system
- Databases & storage
- Monitoring & observability
- Web analytics

Machine Learning & AI ✓

IMAGE

**tensorflow/tensorflow**
tensorflow

Official Docker images for the machine learning framework TensorFlow (<http://...>)

Pulls50M+

Stars2796

Last Updatedabout 21 hours

IMAGE

**pytorch/pytorch**
pytorch

PyTorch is a deep learning framework that puts Python first.

Pulls10M+

Stars1503

Last Updated12 days

IMAGE

**langchain/langchain** 
LangChain

⚡ Building applications with LLMs through composability ⚡

Pulls50K+

Stars321

Last Updatedover 1 year

IMAGE

**python**  ✓
Docker Official Images

Python is an interpreted, interactive, object-oriented, open-source...

Pulls1B+

Stars10217

Last Updatedabout 13 hours ✓

Trending this week

IMAGE

**ai/qwen3** 
Docker

IMAGE

**arm32v7/redis**
arm32v7

IMAGE

**hylang** 
Docker Official Images

IMAGE

**atlassian/confluence** 
Atlassian



2. Package applications with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows**
- d) Check installation & view Docker commands
- e) Key elements of Docker
- f) Main/core directives of Docker
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container from the image
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to test run





c. Install Docker Desktop on Windows

❑ Docker Desktop is a cross-platform tool, free for developers and small teams.
To install on Windows, we need to ensure:

- Windows 10/11 64-bit with at least 4GB RAM.
- Windows Subsystem for Linux 2 (WSL 2) backend or Hyper-V is enabled.

❑ Installation:

(1) Download the installation file from the Docker Desktop for Windows page, then run the .EXE file and configure the components for the WSL 2 backend. After the installation is complete; (2) Launch Docker Desktop and log in to Docker Hub.

=> Specifically as follows:

c. Install Docker Desktop on Windows

(1) Download Docker Desktop by clicking on the Docker Desktop for Windows button on the page:

<https://docs.docker.com/desktop/windows/install/>

- Screenshot:

Home / Manuals / Docker Desktop / Setup / Install / Windows

Install Docker Desktop on Windows Page options ▼

Docker Desktop terms

Commercial use of Docker Desktop in larger enterprises (more than 250 employees OR more than \$10 million USD in annual revenue) requires a [paid subscription](#).

This page provides download links, system requirements, and step-by-step installation instructions for Docker Desktop on Windows.

[Docker Desktop for Windows - x86_64](#) ✓

[Docker Desktop for Windows - x86_64 on the Microsoft Store](#)

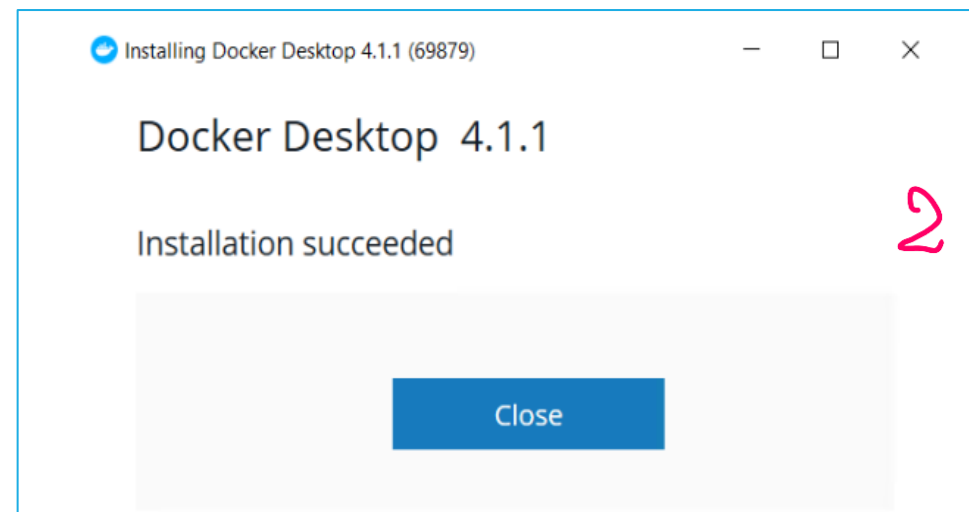
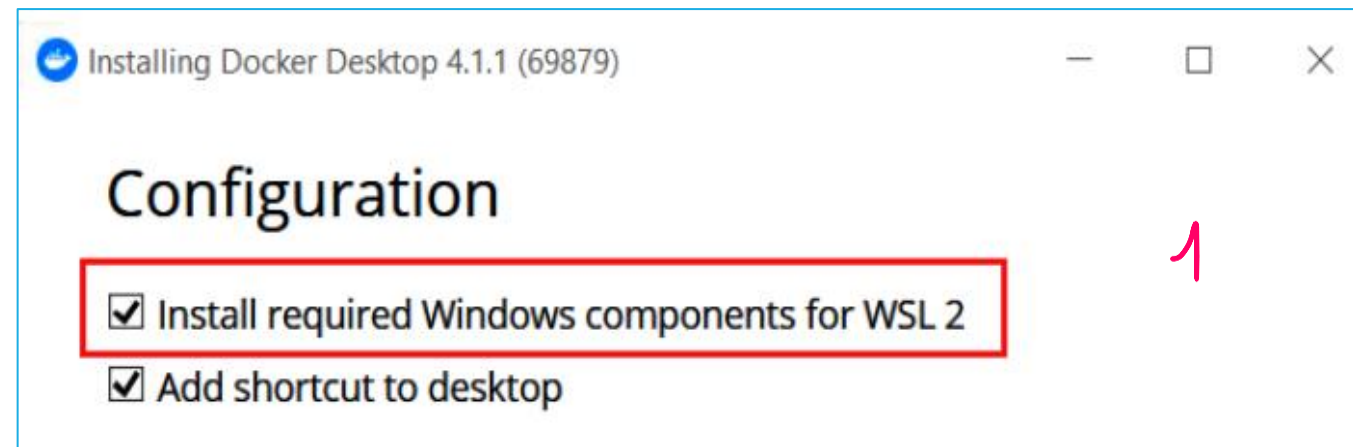
[Docker Desktop for Windows - Arm \(Early Access\)](#)

c. Install Docker Desktop on Windows

❑ Installation & configuration:

⇒ Click on the downloaded executable file (.EXE) to install, during the installation we need to install the components for the WSL 2 backend, by checking the box (as shown in screen 1):

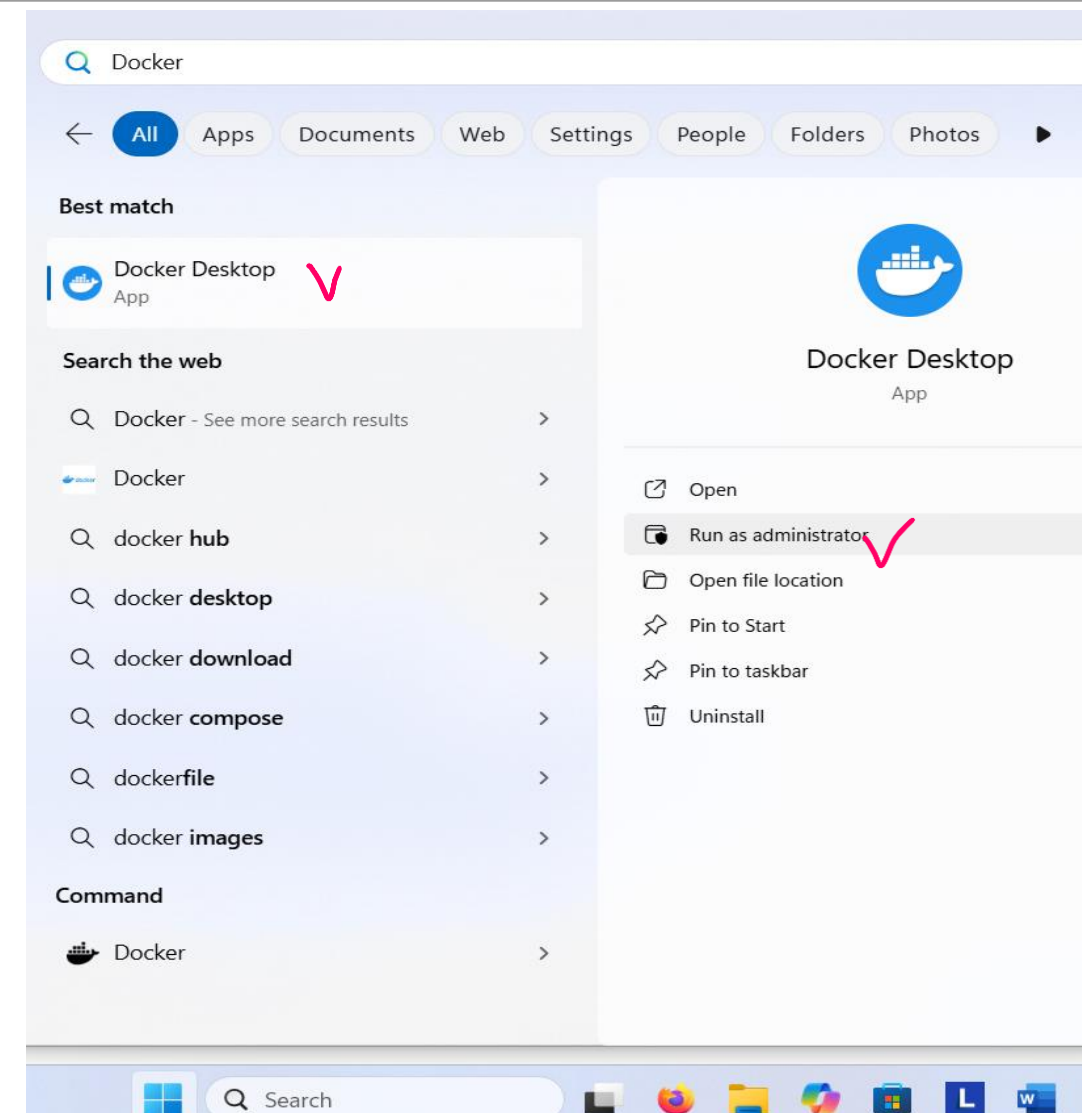
⇒ Successful installation result: screen 2



c. Install Docker Desktop on Windows

(2) Launch Docker Desktop, register & log in to Docker Hub.

- ❑ Startup Docker Desktop as shown in the screenshot:
- ❑ Register & Login to Docker Hub (at the Docker Desktop screen)
=> Result: screenshot (slide below)





Ask Gordon BETA

Containers

Images

Volumes

Builds

Models BETA

Docker Hub

Docker Scout

Extensions

Containers

[Give feedback](#)

View all your running containers and applications. [Learn more](#)

Container CPU usage ⓘ

No containers are running.

Container memory usage ⓘ

No containers are running.

[Show charts](#)

Search



Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	☐	mytomcat	18b3c8aa2b2f	tomcat:9.0	8083:8080	N/A	10 hours ago	▶ ⋮ 🗑️
☐	☐	sonarqube	b2336258c6fa	sonarqube:	9000:9000	N/A	18 hours ago	▶ ⋮ 🗑️
☐	☐	sonardb	6abc93e7a1eb	postgres:15		N/A	18 hours ago	▶ ⋮ 🗑️
☐	☐	jenkins	d9c0e36668b9	jenkins/jenl	50000:50000 Show all ports (2)	N/A	8 hours ago	▶ ⋮ 🗑️
☐	☐	nexus	07778564002f	sonatype/n	8081:8081	N/A	10 hours ago	▶ ⋮ 🗑️
☐	☐	affectionate_khc	233058623aa1	phamthuong	8080:80	N/A	2 months ago	▶ ⋮ 🗑️
☐	☐	fervent_hodgkin	c9cb81db092a	phamthuong	8080:8080	N/A	2 months ago	▶ ⋮ 🗑️
☐	☐	demoapp	95993c324cf3	demobook:	8080:80			



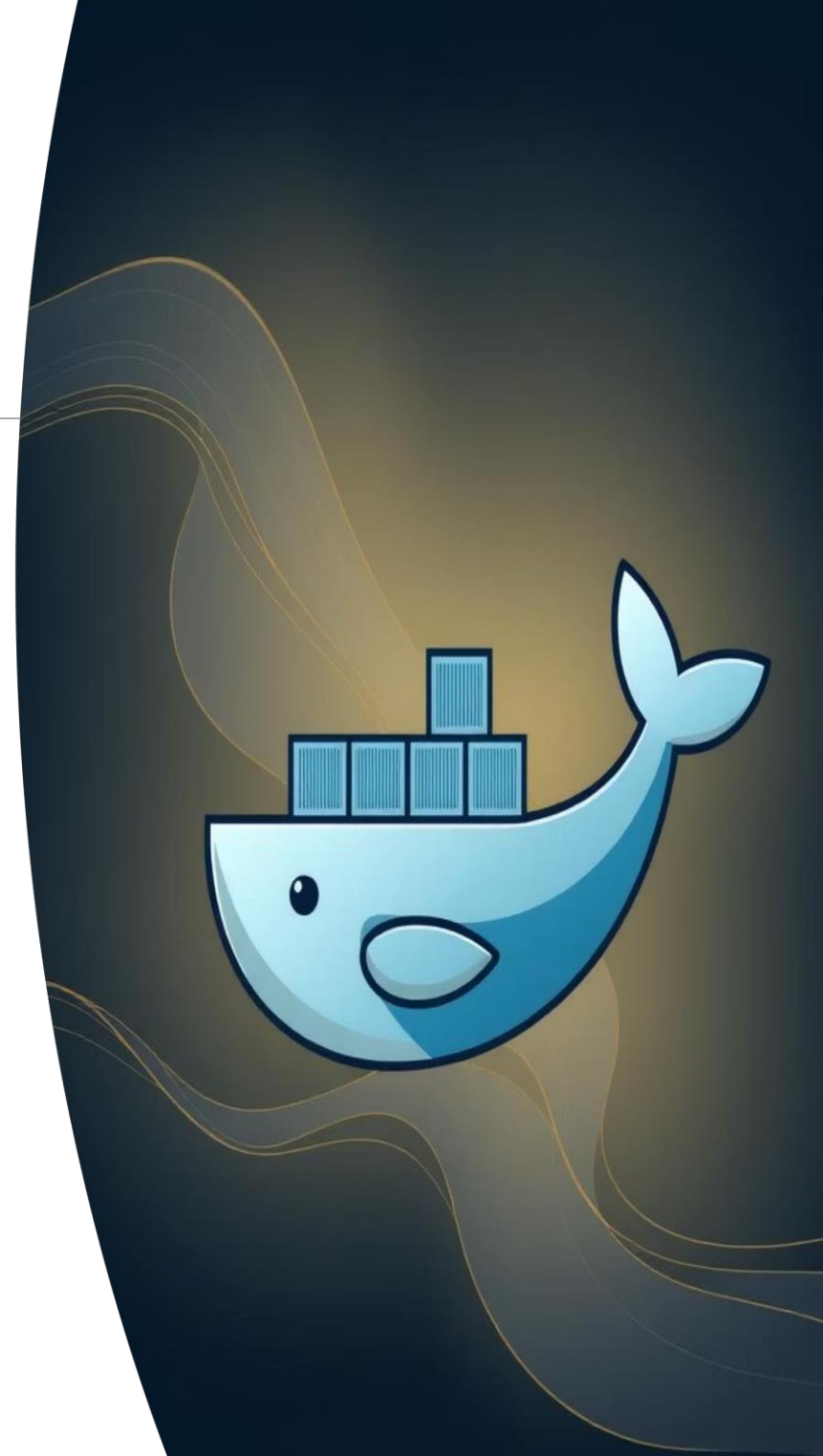
You're now signed in as
ptthuong@ictu.edu.vn





2. Package applications with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands**
- e) Key elements of Docker
- f) Main/core directives of Docker
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container from the image
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to test run





d. Check installation & view Docker commands

- ❑ To see the Docker commands, open DOS, or PowerShell and execute the command:

docker --help

- ❑ To check the installed Docker version, run the command:

docker --version

```
C:\WINDOWS\system32\cmd.  X  +  v

C:\Users\FPTSHOP>docker --version
Docker version 28.1.1, build 4eba377

C:\Users\FPTSHOP>docker --help
Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

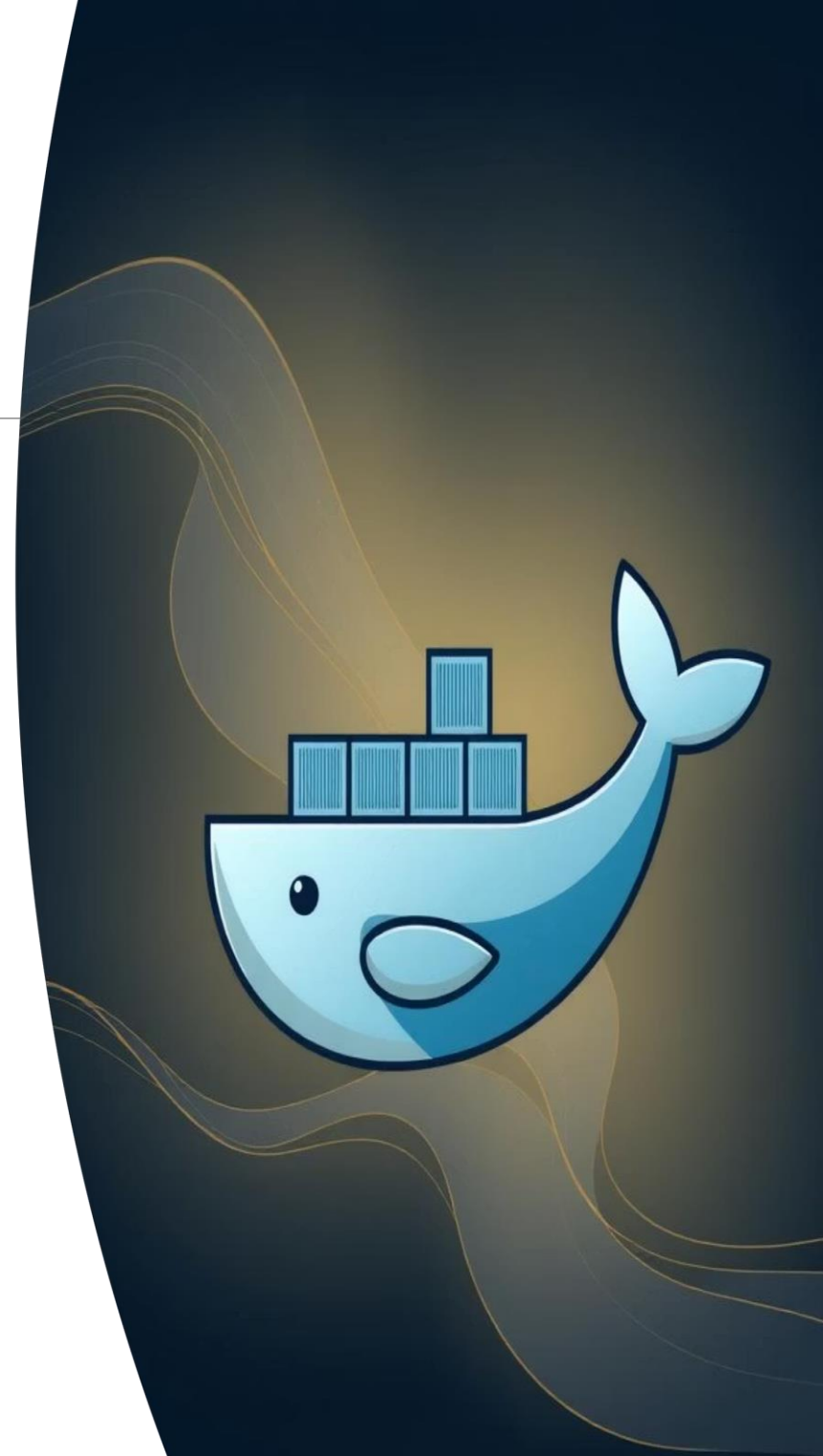
Common Commands:
  run          Create and run a new container from an image
  exec        Execute a command in a running container
  ps          List containers
  build       Build an image from a Dockerfile
  bake        Build from a file
  pull        Download an image from a registry
  push        Upload an image to a registry
  images      List images
  login        Authenticate to a registry
  logout      Log out from a registry
  search       Search Docker Hub for images
  version      Show the Docker version information
  info        Display system-wide information

Management Commands:
  ai*          Docker AI Agent - Ask Gordon
  builder      Manage builds
  buildx*      Docker Buildx
  cloud*       Docker Cloud
  compose*     Docker Compose
  container    Manage containers
```



2. Packaging applications with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands
- e) Key elements of Docker**
- f) Main/core directives of Docker
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container from the image
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to test run





e. Key elements of Docker

Before learning Docker commands, we need to familiarize ourselves with the concepts: Docker images, Dockerfiles, containers, and volumes, ... to understand the elements of Docker

1. Docker Image

A basic element of Docker, created from a Dockerfile, containing the binaries and application files to be packaged.

2. Docker Registry

A centralized storage system for shared Docker images, which can be public (Docker Hub) or private (ACR).

3. Container

An instance executed from a Docker image, where the application will run. There can be multiple instances from the same image.

4. Volume

Physical storage space on the host OS, outside the container, that can be shared between multiple containers to store data.

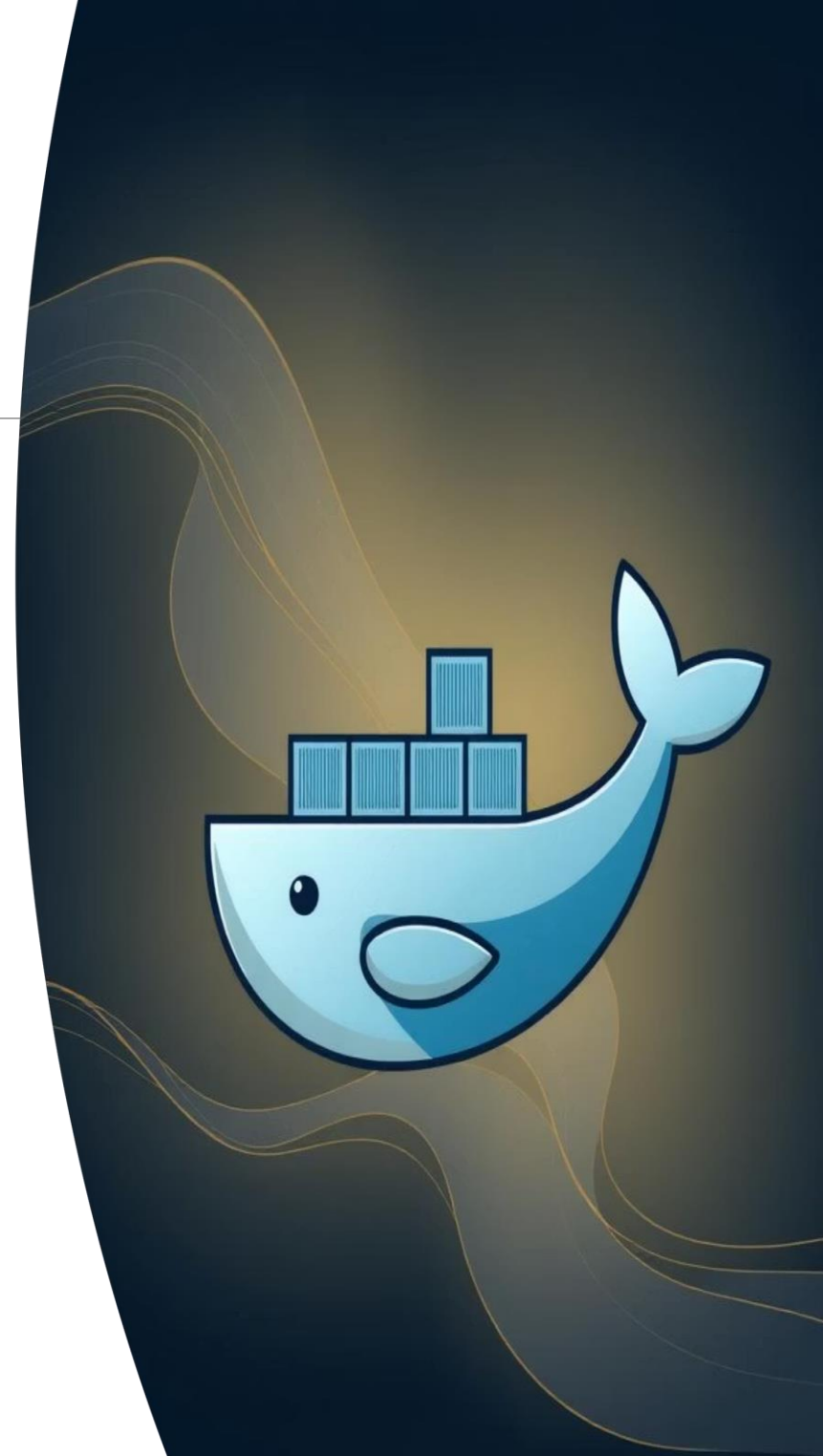
5. Dockerfile

A basic element of Docker, containing instructions for building a Docker image.



2. Packaging applications with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands
- e) Key elements of Docker
- f) Main/core directives of Docker**
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container from the image
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to test run





f. Main/core directives of Docker

1. FROM directive:

- Used to specify the base image that the container will inherit.
- Each Dockerfile must start with at least one FROM.
- Example: FROM openjdk:11 => Use the openjdk:11 image as a base to build a container to run the Java application.

2. COPY directive:

- Used to copy files/folders from the host machine (where you write the Dockerfile) into the container.
- Example: COPY target/myapp.war /usr/local/tomcat/webapps/ => Copy the myapp.war file from the target folder on the host machine to the webapps folder in the Tomcat container.



f. Main/core directives of Docker

3. ADD directive:

- Similar to COPY but more powerful (Can automatically extract .tar files, Supports URLs to download files from the Internet)
- Example: ADD `https://example.com/app.tar.gz /app/` => Download the app.tar.gz file from the Internet, copy it to /app/ in the container and automatically extract it.

4. RUN directive:

- Used to run commands during the image build process, usually to install packages, set up the environment.
- Example: RUN `apt-get update && apt-get install -y curl` => Update the package list and install the curl tool into the image.



f. Main/core directives of Docker

5. ENV directive:

- Used to declare environment variables in the container.
- Example: ENV APP_HOME=/usr/local/myapp => Create the APP_HOME environment variable. Then it can be used in the Dockerfile or when the container is running

6. WORKDIR directive:

- Sets the default working directory in the container. Subsequent commands (RUN, CMD, COPY, ...) will be executed in this directory.
- Example: dockerfile:

```
WORKDIR /app  
COPY ..  
RUN javac Main.java  
CMD ["java", "Main"]
```



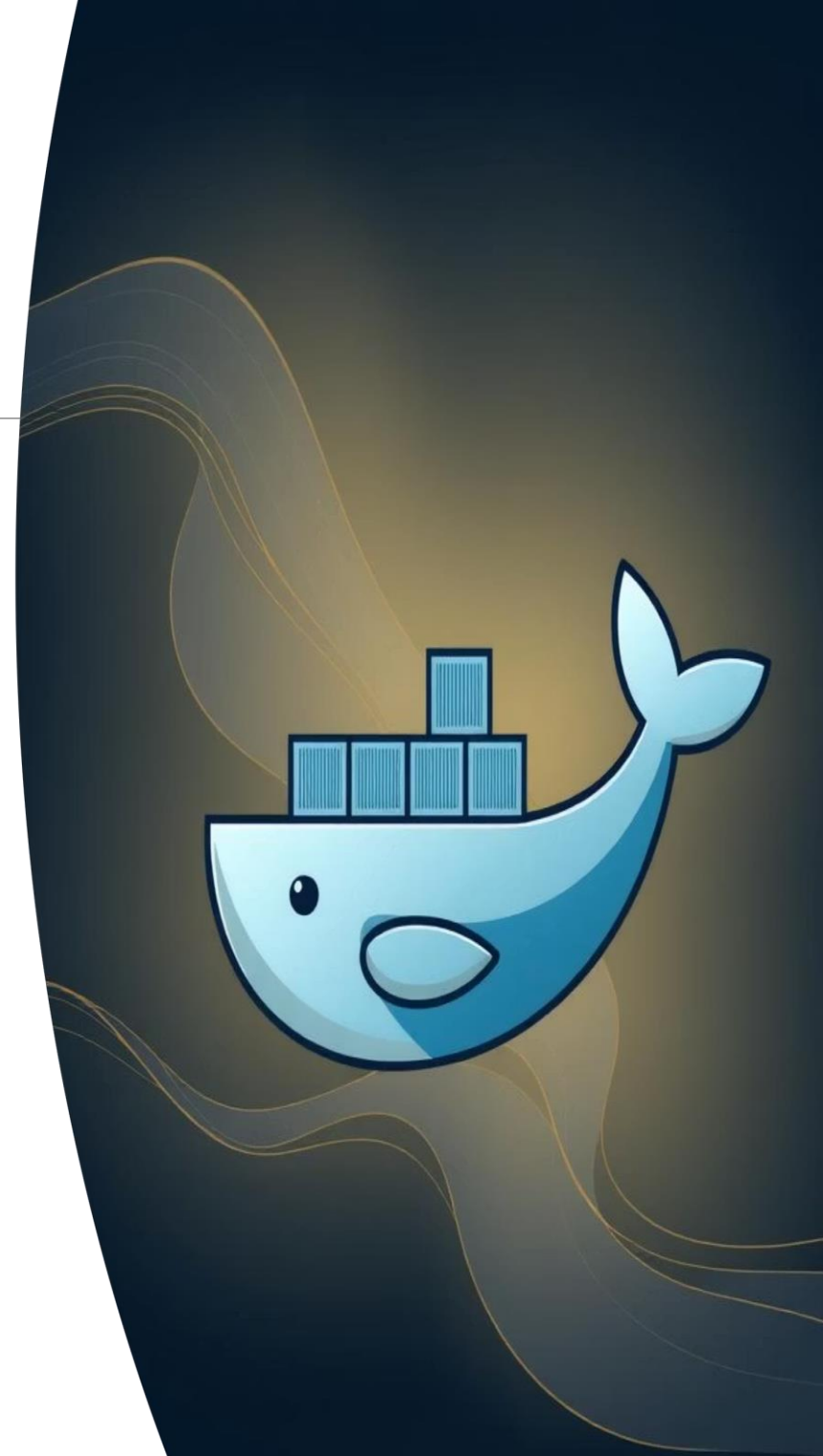
Meaning:

- Set the default working directory to /app.
- Copy all source code to /app.
- Compile the Main.java file.
- When the container runs, execute java Main.



2. Packaging Applications with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check Installation & View Docker Commands
- e) Key Elements of Docker
- f) Main/Core Directives of Docker
- g) **Create Dockerfile with Basic Directives**
- h) Building a Docker image from Dockerfile
- i) Initialize a New Container from the Image
- j) Push the Built Image to Docker Hub
- k) Pull the Image from Docker Hub to Test Run





g. Create Dockerfile with Basic Directives

❑ Example of building a Docker image (containing an Apache web server and a web application)

1. Create a simple web application:

⇒ Create a folder named appdocker, containing the index.html file, for example with the following content:

```
<html>
  <body>
    <h1>Welcome to my new app</h1>
    This page is test for my demo Dockerfile.<br />
    Enjoy ...
  </body>
</html>
```

2. Then, in the same folder, create a Dockerfile (without extension) with the content (slide below):



g. Create Dockerfile with Basic Directives

2) Dockerfile content:

```
FROM httpd:latest  
COPY index.html /usr/local/apache2/htdocs/
```

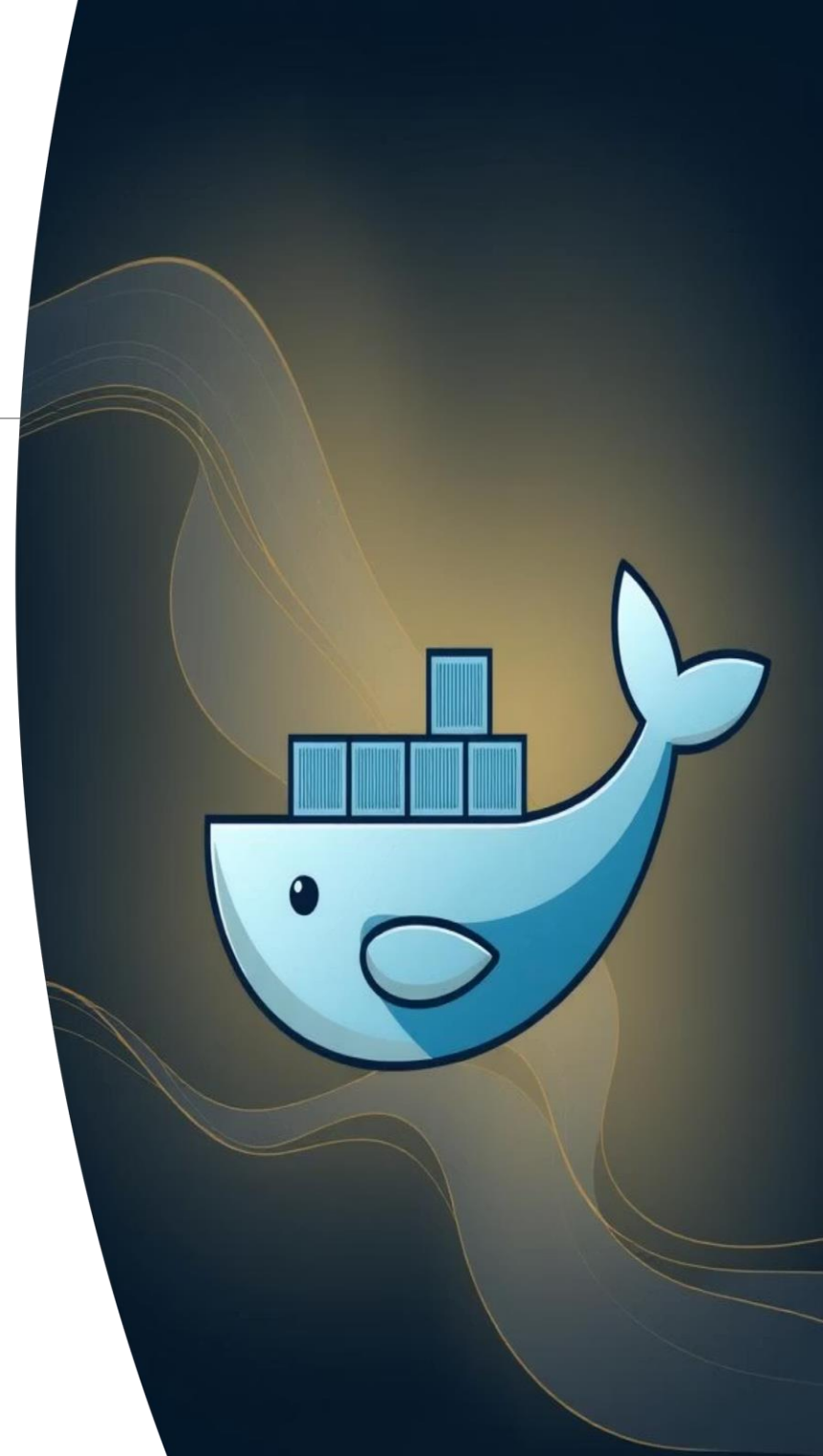
Where:

- FROM Directive: specifies the base image that the container will inherit => httpd (latest version) is the base image in this example.
=> httpd is the official base image of Apache HTTP Server (the most popular web server in the world) => helps to quickly launch an Apache web server in a container (used to deploy static web applications (HTML, CSS, JS))
- COPY Directive: executes the image building process. Docker will copy the local index.html file created into the /usr/local/apache2/htdocs/ directory of the image.



2. Packaging Applications with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check Installation & View Docker Commands
- e) Key Elements of Docker
- f) Main/Core Directives of Docker
- g) Create Dockerfile with Basic Directives
- h) Building a Docker image from Dockerfile**
- i) Initialize a New Container from the Image
- j) Push the Built Image to Docker Hub
- k) Pull the Image from Docker Hub to Test Run



h. Building a Docker image from Dockerfile

~ package application using Docker

- ❑ Open a terminal (PowerShell) and navigate to the directory containing the Dockerfile, then execute the docker build command as follows:

docker build -t demobook:v1 .

- Where:

- ✓ The -t argument specifies the name of the image and its tag. In this example, the name of the image is demobook and its tag is v1.
- ✓ The . (dot) at the end of the command specifies that we will use the files in the current directory.

=> Screenshot of command execution:

```
PS C:\Users\FPTSHOP> docker --version
Docker version 28.1.1, build 4eba377
PS C:\Users\FPTSHOP> cd D:\Namhoc_2025\Hocky_I\BaoTriPM\HoclieuLCMS_K21\demo\bai8_demo_docker\appdocker
PS D:\Namhoc_2025\Hocky_I\BaoTriPM\HoclieuLCMS_K21\demo\bai8_demo_docker\appdocker> docker build -t demobook:v1 .
[+] Building 9.0s (7/7) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile             0.0s
=> => transferring dockerfile: 100B                             0.0s
=> [internal] load metadata for docker.io/library/httpd:latest 3.8s
=> [internal] load .dockerignore                               0.0s
=> => transferring context: 2B                                    0.0s
=> [internal] load build context                               0.1s
=> => transferring context: 177B                                  0.0s
=> [1/2] FROM docker.io/library/httpd:latest@sha256:1ae8051591a5ded56e4a3d7399c423e940e8475ad0e5adb82e6e10893fe9 4.9s
=> => resolve docker.io/library/httpd:latest@sha256:1ae8051591a5ded56e4a3d7399c423e940e8475ad0e5adb82e6e10893fe9 0.0s
=> => sha256:816d153af128394da136780110978a27fabf1ae1062d65799165cfcaeffe1de9 293B / 293B 0.7s
=> => sha256:d812016dda12d10a4887d1c8c047edf1bdd3e77f095fb390e77c6e86aea7a4b4 26.06MB / 26.06MB 2.6s
=> => sha256:365277d54dac631554fd8799874c2798cf00241efd99758e8a3b81d551f91600 4.21MB / 4.21MB 2.3s
=> => sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 32B / 32B 1.1s
=> => sha256:1a50d37c990cc2ea20582da2b22b8b2a84593c973606d19392ed8a848a3e7eed 145B / 145B 0.5s
=> => sha256:3da95a905ed546f99c4564407923a681757d89651a388ec3f1f5e9bf5ed0b39d 28.23MB / 28.23MB 2.8s
=> => extracting sha256:3da95a905ed546f99c4564407923a681757d89651a388ec3f1f5e9bf5ed0b39d 0.5s
=> => extracting sha256:1a50d37c990cc2ea20582da2b22b8b2a84593c973606d19392ed8a848a3e7eed 0.0s
=> => extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1 0.0s
=> => extracting sha256:365277d54dac631554fd8799874c2798cf00241efd99758e8a3b81d551f91600 0.1s
=> => extracting sha256:d812016dda12d10a4887d1c8c047edf1bdd3e77f095fb390e77c6e86aea7a4b4 0.3s
=> => extracting sha256:816d153af128394da136780110978a27fabf1ae1062d65799165cfcaeffe1de9 0.0s
=> [2/2] COPY index.html /usr/local/apache2/htdocs/ (2) 0.1s
=> exporting to image 0.1s
=> => exporting layers 0.0s
=> => exporting manifest sha256:c1efeb1aace40d624d916e52539511369a36ac17cf85206eabf4680452c4c40e (3) 0.0s
```

h. Building a Docker image from Dockerfile

- ❑ From the running screen, we see that building a Docker image consists of 3 steps:
 - 1) Docker downloads the specified base image (from Docker Hub) (1)
 - 2) Docker copies the index.html file into the image (2)
 - 3) Docker creates and tags the image (3)

Note:

- If during the execution of the docker build command you encounter the message:
`https://registry-1.docker.io/v2/library/httpd/manifests/latest: unauthorized:`
incorrect username or password error, try executing the docker logout command.
Then, re-execute the docker build command.

h. Building a Docker image from Dockerfile

- ❑ To check if the image has been created successfully, execute the command:

docker images

=> Screenshot: created successfully

```
=> => exporting manifest list sha256:237be006655281f0136420f03645521c170d0c0101d3762220055210dc5c00dd 0.0s
=> => naming to docker.io/library/demobook:v1 0.0s
=> => unpacking to docker.io/library/demobook:v1 0.0s
PS D:\Namhoc_2025\Hocky_I\BaoTriPM\HoclieuLCMS_K21\demo\bai8_demo_docker\appdocker> docker images
REPOSITORY    TAG          IMAGE ID      CREATED       SIZE
demobook      v1          237be0066552 About a minute ago 221MB
PS D:\Namhoc_2025\Hocky_I\BaoTriPM\HoclieuLCMS_K21\demo\bai8_demo_docker\appdocker>
```


h. Building a Docker image from Dockerfile

Note:

- To build a Docker image from a .jar file (Spring Boot or Java app), the Dockerfile will be as follows:
 - ✓ Write Dockerfile (screenshot)
 - ✓ After having the Dockerfile, run the image build command: `docker build -t myapp:1.0` Explanation:
 - ✓ **docker build: image build command**
 - ✓ **-t myapp:1.0: attach a tag to the image (name is myapp, version 1.0)**
 - ✓ **.: dot (.) ~ build context, meaning Docker takes the Dockerfile in the current directory**

```
# Use JDK runtime (only need JRE if not building in the
container)
FROM openjdk:17-jdk-slim

# Set the working directory in the container
WORKDIR /app

# Copy the jar file from the host machine to the
container
COPY myapp.jar app.jar

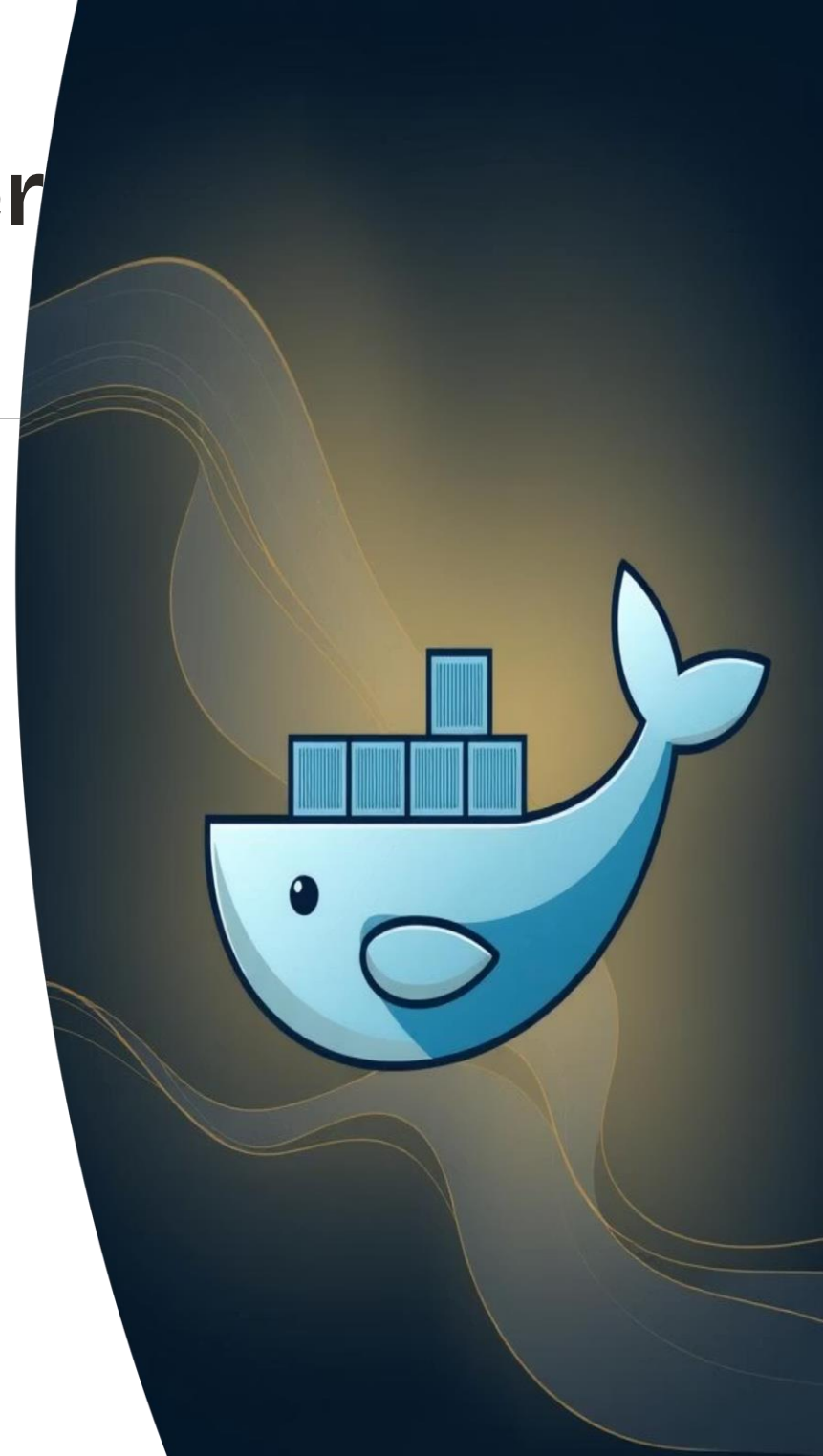
# Open the application port (e.g., 8080)
EXPOSE 8080

# Command to run when the container starts
ENTRYPOINT ["java", "-jar", "app.jar"]
```



2. Packaging the application with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands
- e) Key elements of Docker
- f) Main/core directives of Docker
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container of the image**
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to test run





i. Initialize a new Container of the image

- ❑ After building the Docker image for the application, we initialize a new container to test run the application, execute the command:

`docker run -d --name demoapp -p 8080:80 demobook:v1`

○ **Where:**

- ✓ **`-d`**: ~ container will be in background mode
- ✓ **`--name`**: set container name,
- ✓ **`-p`** port mapping, port 80 of the container will be mapped to port 8080 on the host (our PC)
- ✓ **`demobook:v1`**: name of the image and its tag

```
PS D:\Namhoc_2025\Hocky_I\BaoTriPM\HoclieuLCMS_K21\demo\bai8_demo_docker\appdocker> docker run -d --name demoapp -p 8080:80 demobook:v1
95993c324cf3524400568d970f21dd194c3e1a51d9cb7228a648d6a88993a0f5
PS D:\Namhoc_2025\Hocky_I\BaoTriPM\HoclieuLCMS_K21\demo\bai8_demo_docker\appdocker> .
```



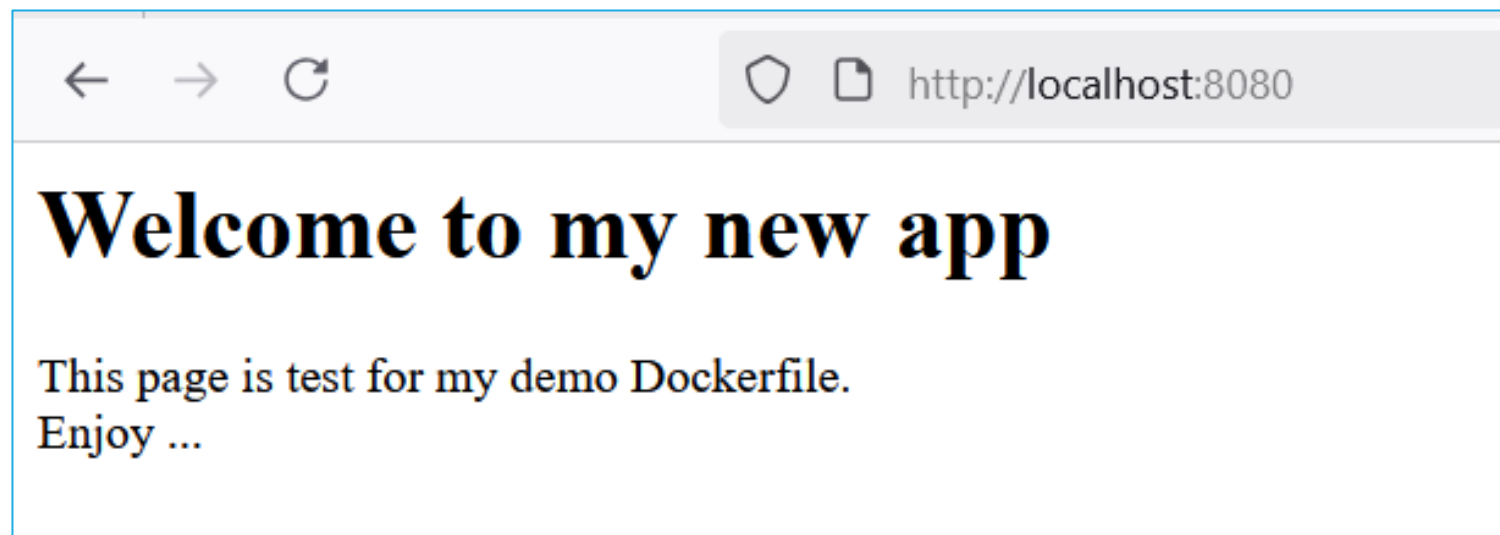
i. Launch a new container of the image

- ❑ Check the list of running containers using docker ps

```
PS C:\Users\mkrief> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS ✓	PORTS
6cce2099b174	demobook:v1	"httpd-foreground"	2 weeks ago	<u>Up</u> About a minute	0.0.0.0:8080->80/tcp

=> Everything in the container is isolated, but thanks to port mapping, we can access the application via the browser at `http://localhost:8080` => Result: The content of the index.html page is displayed.

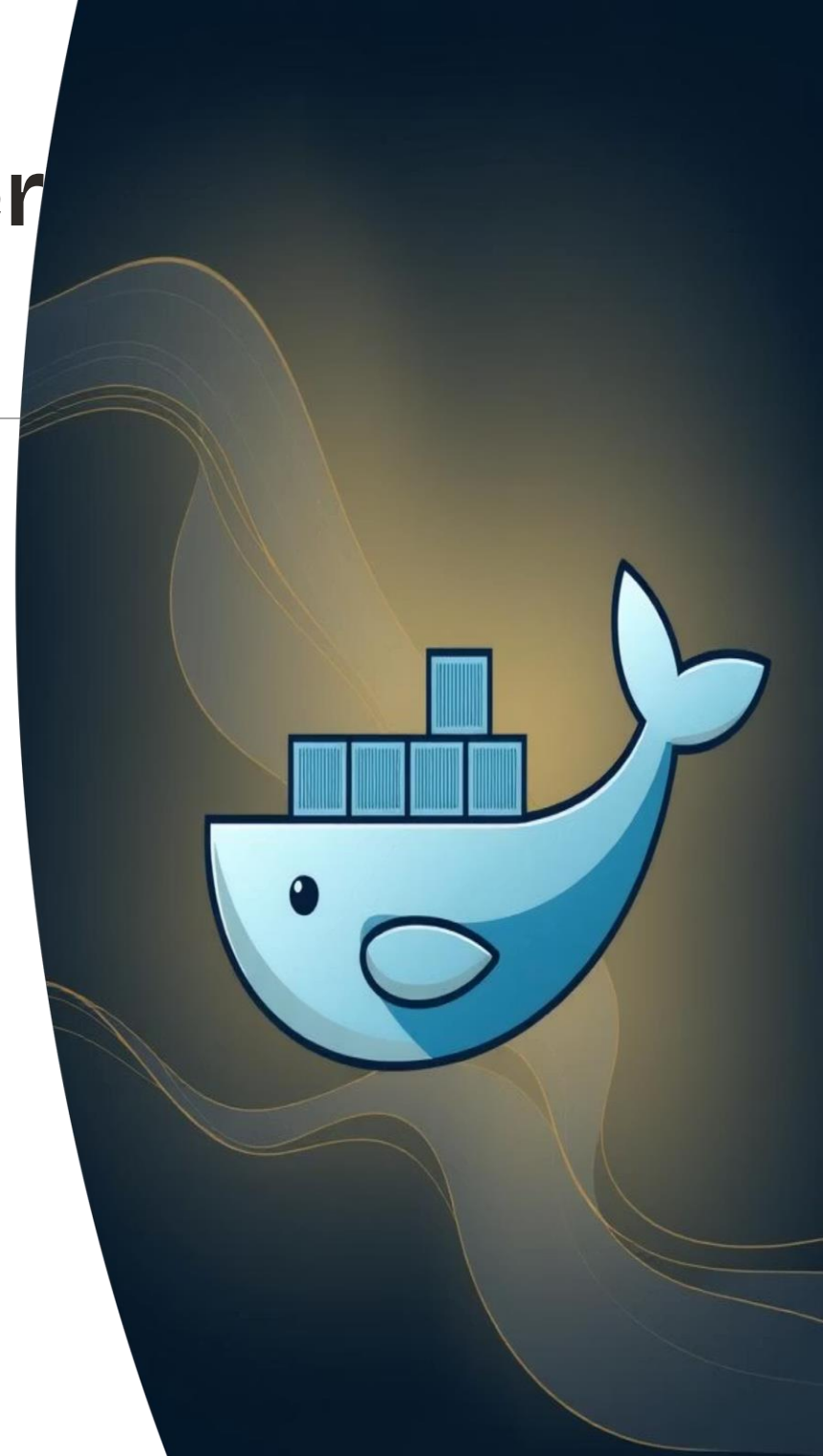


Press refresh to test again after stopping the application container.



2. Packaging the application with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands
- e) Key elements of Docker
- f) Main/core directives of Docker
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container of the image
- j) Push the built Image to Docker Hub**
- k) Pull the Image from Docker Hub to test run





j. Push Image to Docker Hub

- ❑ To share and use Docker images on other computers, we need to push it to a Docker image registry. Docker Hub is a public, free registry.
 - We have registered for Docker Hub (according to the instructions in the previous slides)
=> Screenshot:

The screenshot shows the Docker Hub web interface. The top navigation bar is blue with the 'hub' logo, 'Explore' link, and 'My Hub' link (which is underlined and has a red checkmark). To the right of the navigation bar is a search bar labeled 'Search Docker Hub' and a 'CtrlK' button. Further right are icons for help, notifications, and a user profile icon (a circle with 'P' and a red checkmark). On the left side, there is a sidebar menu with a dropdown for 'phamthuongdocker Docker Personal'. The menu items are 'Repositories' (highlighted), 'Collaborations', 'Settings', and 'Default privacy'. The main content area is titled 'Repositories' and shows 'All repositories within the phamthuongdocker namespace.' Below this is a search bar 'Search by repository name' and a dropdown menu 'All content'. On the right side of the main content area is a blue button 'Create a repository'. At the bottom of the main content area, there is a table header with columns: 'Name', 'Last Pushed' (with an upward arrow), 'Contains', 'Visibility', and 'Scout'.



j. Push Image to Docker Hub

1. Log in to Docker Hub

Use the command `docker login -u <your dockerhub login>` and enter your password.

2. Get Image ID

Run the `docker images` command to display a list of images and their IDs.

3. Tag Image for Docker Hub

Use the image ID to tag: `docker tag <image ID> <dockerhub login>/demobook:v1`

4. Push Image to Docker Hub

Execute the command `docker push docker.io/<dockerhub login>/demobook:v1` => The image will be uploaded to Docker Hub and can be viewed on the web portal.

=> See the push results in the screenshot (slide below)

 Explore My Hub 

Search Docker Hub

 **phamthuongdocker**
Docker Personal

Repositories 

Collaborations

Settings

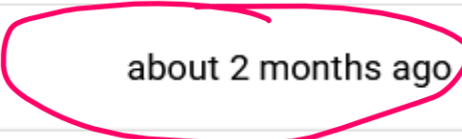
Default privacy

Notifications

Billing

Repositories

All repositories within the **phamthuongdocker** namespace.

Name	 Last Pushed 
phamthuongdocker/demobook 	 about 2 months ago

```
Login Succeeded
PS D:\Namhoc_2025\Hocky_I\BaoTriPM\HoclieuLCMS_K21\demo\bai8_demo_docker\appdocker> docker push docker.io/
The push refers to repository [docker.io/phamthuongdocker/demobook]
934a49fb6598: Pushed
4f4fb700ef54: Pushed
3da95a905ed5: Pushed
1a50d37c990c: Pushed
d812016dda12: Pushed
816d153af128: Pushed
365277d54dac: Pushed
4afff44118d6: Pushed
v1: digest: sha256:237be00665528ff6f9042afa5b43321cf78d6e6f0fd578222069521bae9e06ad size: 856
PS D:\Namhoc_2025\Hocky_I\BaoTriPM\HoclieuLCMS_K21\demo\bai8_demo_docker\appdocker> 
```



j. Push Image to Docker Hub

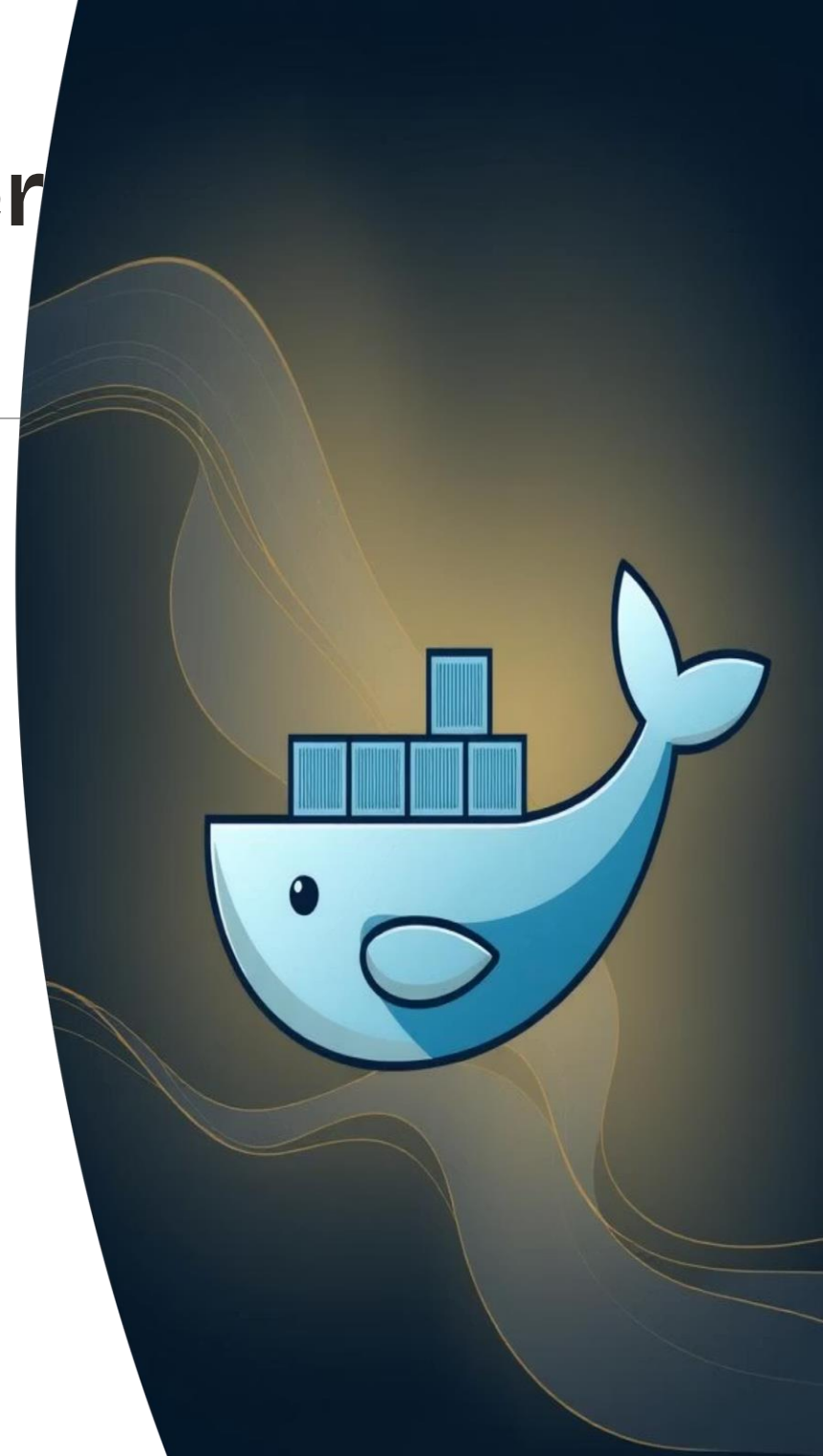
❑ By default, the image pushed to Docker Hub is in public mode – everyone can view it in the explorer and use it.

=> It is possible to create a private image – only when we confirm can the image be used (refer to textbook [1])



2. Packaging the application with Docker

- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands
- e) Key elements of Docker
- f) Main/core directives of Docker
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container of the image
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to test run**





k. Pull the Image from Docker Hub to test run

Stop the running demobook:v1 container (previously), pull the image from Docker Hub, & test run

1. View the list of running containers: use the `docker ps` command

2. Stop the container: run the `docker stop <Container ID>` command

3. Pull the Image from Docker Hub: run the `docker pull phamthuongdocker/demobook:v1` command

4. Run the container from the image

Execute the command `docker run -d -p 8080:80
phamthuongdocker/demobook:v1`

5. Check the application corresponding to the running container

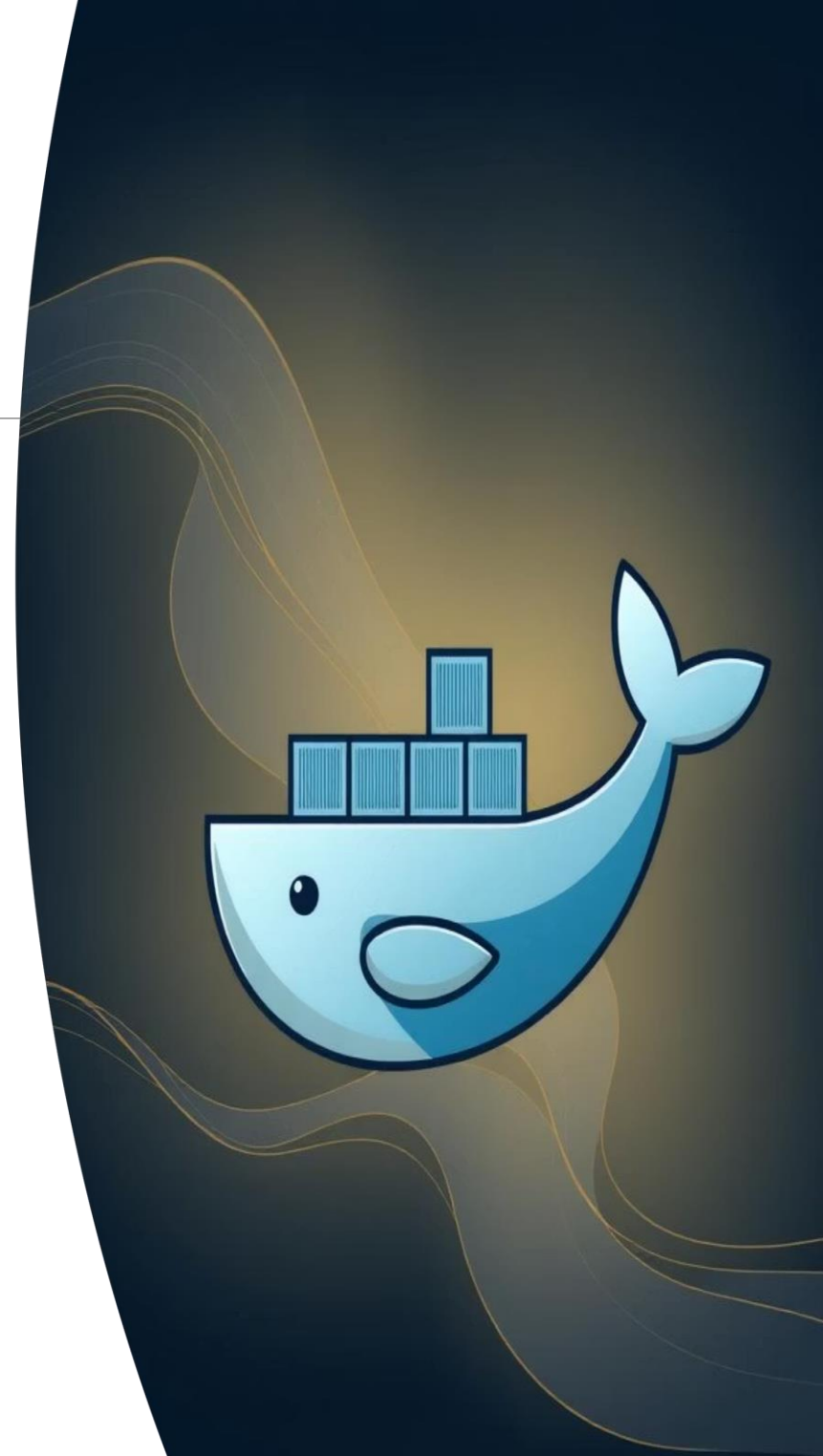
Access: `http://localhost:8080`

Done!



2. Package the application with Docker

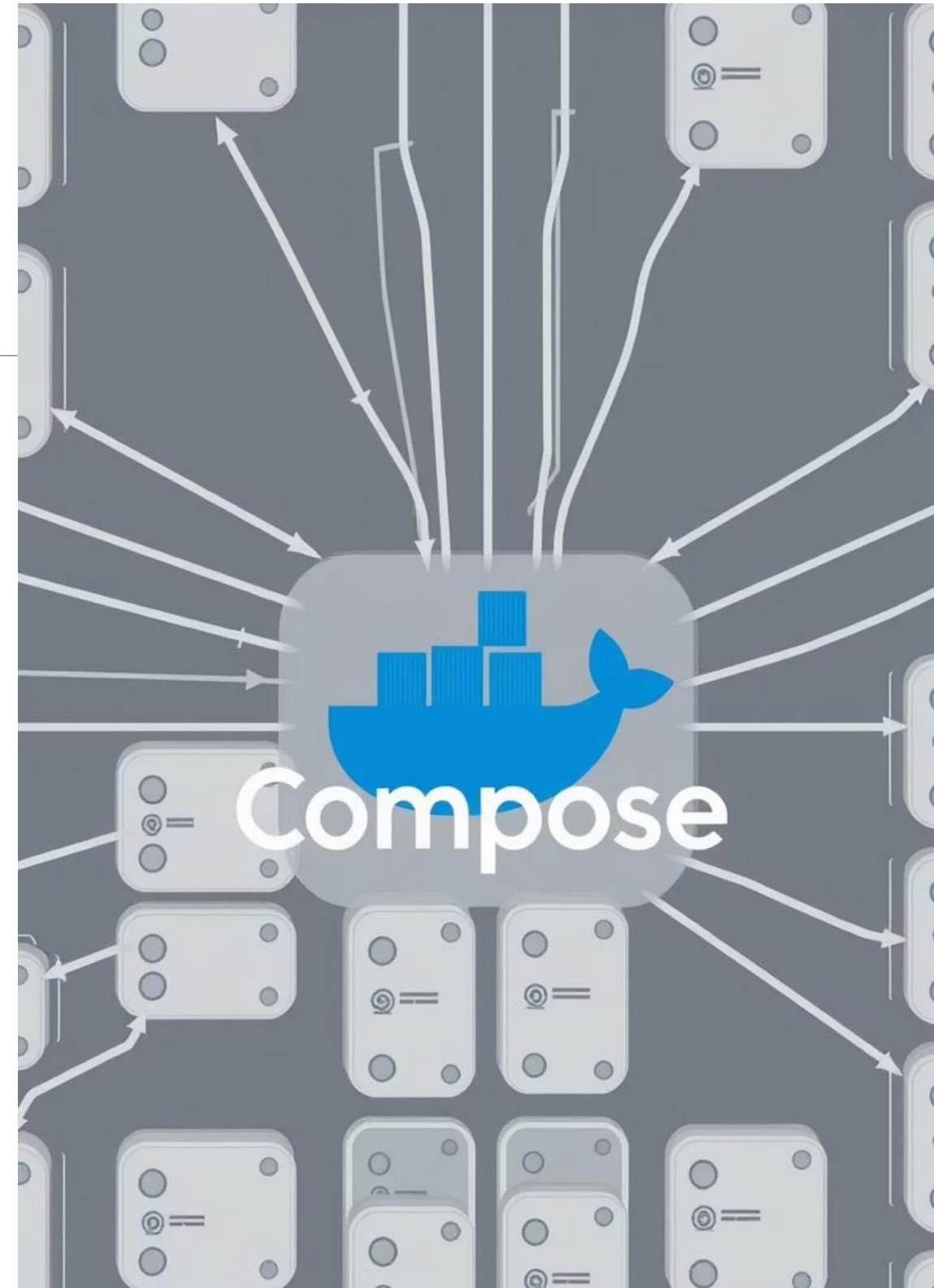
- a) Introduction
- b) Register & Explore Docker Hub
- c) Install Docker Desktop on Windows
- d) Check installation & view Docker commands
- e) Key elements of Docker
- f) Main/core directives of Docker
- g) Create Dockerfile with basic directives
- h) Building a Docker image from Dockerfile
- i) Initialize a new Container of the image
- j) Push the built Image to Docker Hub
- k) Pull the Image from Docker Hub to test run



3. Docker Compose

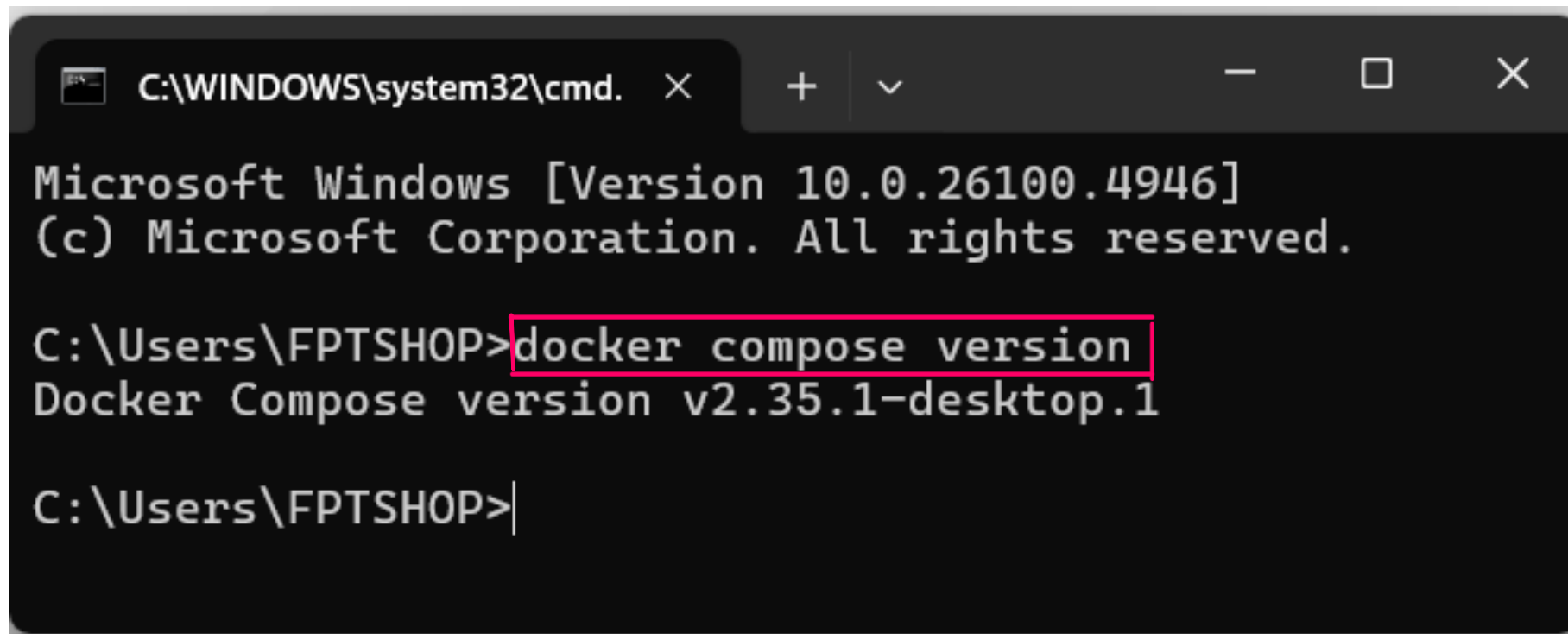
- ❑ Modern applications rarely operate independently, often requiring dependent services such as databases or APIs.
 - **Docker Compose allows defining and managing multiple containers in the same application.**
 - Docker Compose uses a YAML format configuration file to describe the services, volumes, and networks that make up the application
 - That is, instead of running each complex docker run ... command, we only need to write a docker-compose.yml file describing the services (applications), then use the command: docker compose up
 - → Docker will automatically build, create a network, and launch all containers.

Thus: Docker Compose = Managing multi-container applications (Microservices, Database + Backend + Frontend).



3. Docker Compose

- ❑ Docker Compose is available in Docker Desktop, we can check with the command:
`docker compose version`



```
C:\WINDOWS\system32\cmd. X + v - □ X

Microsoft Windows [Version 10.0.26100.4946]
(c) Microsoft Corporation. All rights reserved.

C:\Users\FPTSHOP>docker compose version
Docker Compose version v2.35.1-desktop.1

C:\Users\FPTSHOP>|
```



3. Docker Compose – Usage Process

1. Create a docker-compose.yml file to define the services.
2. Execute Docker compose to start the containers described in the YAML configuration file, run the docker-compose up -d command in the directory containing the file
 - The -d option runs the containers in detached mode, background mode.
 - You can verify the running containers with the docker ps command, or through Docker Desktop
2. Container lifecycle management:
 - Stop: docker compose down
 - View log: docker compose logs -f
 - Restart: docker compose restart



3. Docker Compose – Student Practice

Docker Compose for Java web application + MySQL + Tomcat.

□ Practice, discussion:

- Suppose we have a Java web application (Maven packages to myapp.war file) and want to run it on Tomcat. The application needs to connect to the MySQL database.
 - So we will have 2 services in Docker Compose:
 - ✓ **tomcat** → **contains Java web application (deploy .war).**
 - ✓ **mysql** → **database backend.**
 - What is the content of the docker-compose.yml file????

Summary & Reinforcement Questions

This lesson has covered the basic concepts of Docker, from creating Dockerfiles and images to deploying containers locally and to Docker Hub. We've also explored how to use Docker Compose to manage multiple containers.

Key Concepts

- Docker Hub and Docker Desktop
- Dockerfile and Directives
- `docker build`, `run`, `push` commands
- Docker Compose

Reinforcement Questions

- What is Docker Hub?
- Which directive defines the base image in a Dockerfile?
- Which command creates a Docker image?
- Which command publishes an image to Docker Hub?
- How does CD differ from continuous deployment?



Practice, discussion

JOB #3 & JOB #4 PRACTICE:
[SEE HERE](#)



Thank you!

THAI NGUYEN, MONTH ... YEAR 2025
